



**Sir Syed CASE
Institute of Technology**

Assignment #3
Artificial Intelligence

SUBMITTED BY:

UBAID AHMAD

ROLL NO:

2410-0011

SUBMITTED TO:

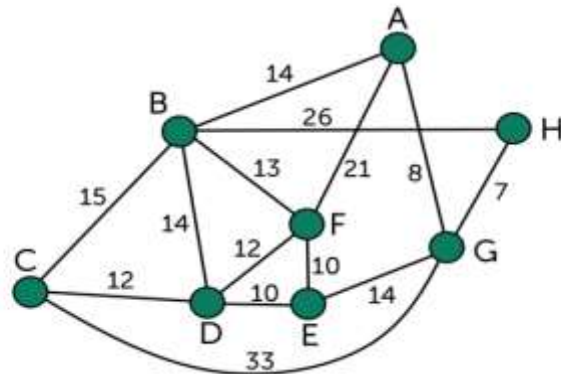
Sir Hamid Ashfaq

DATE:

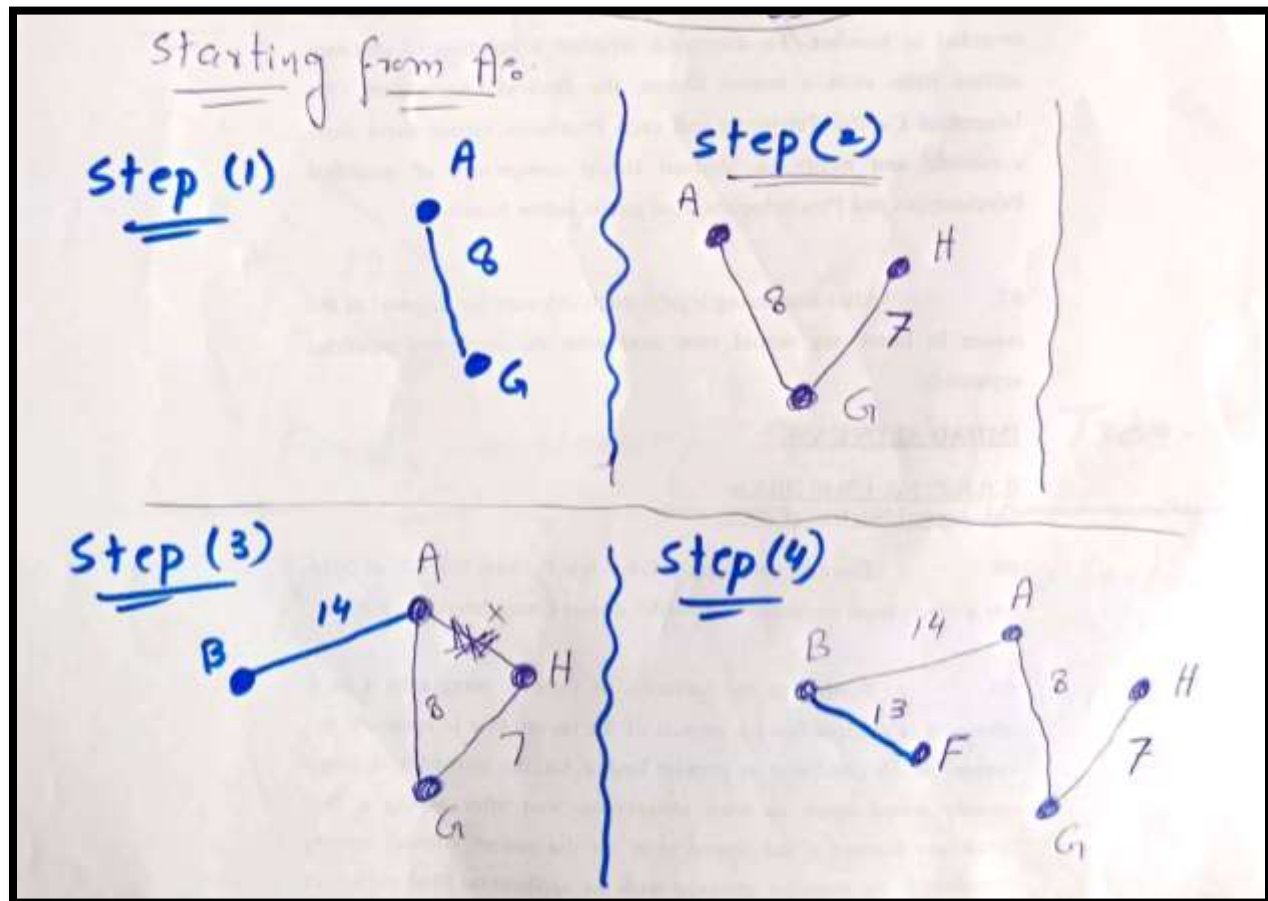
04/05/2026

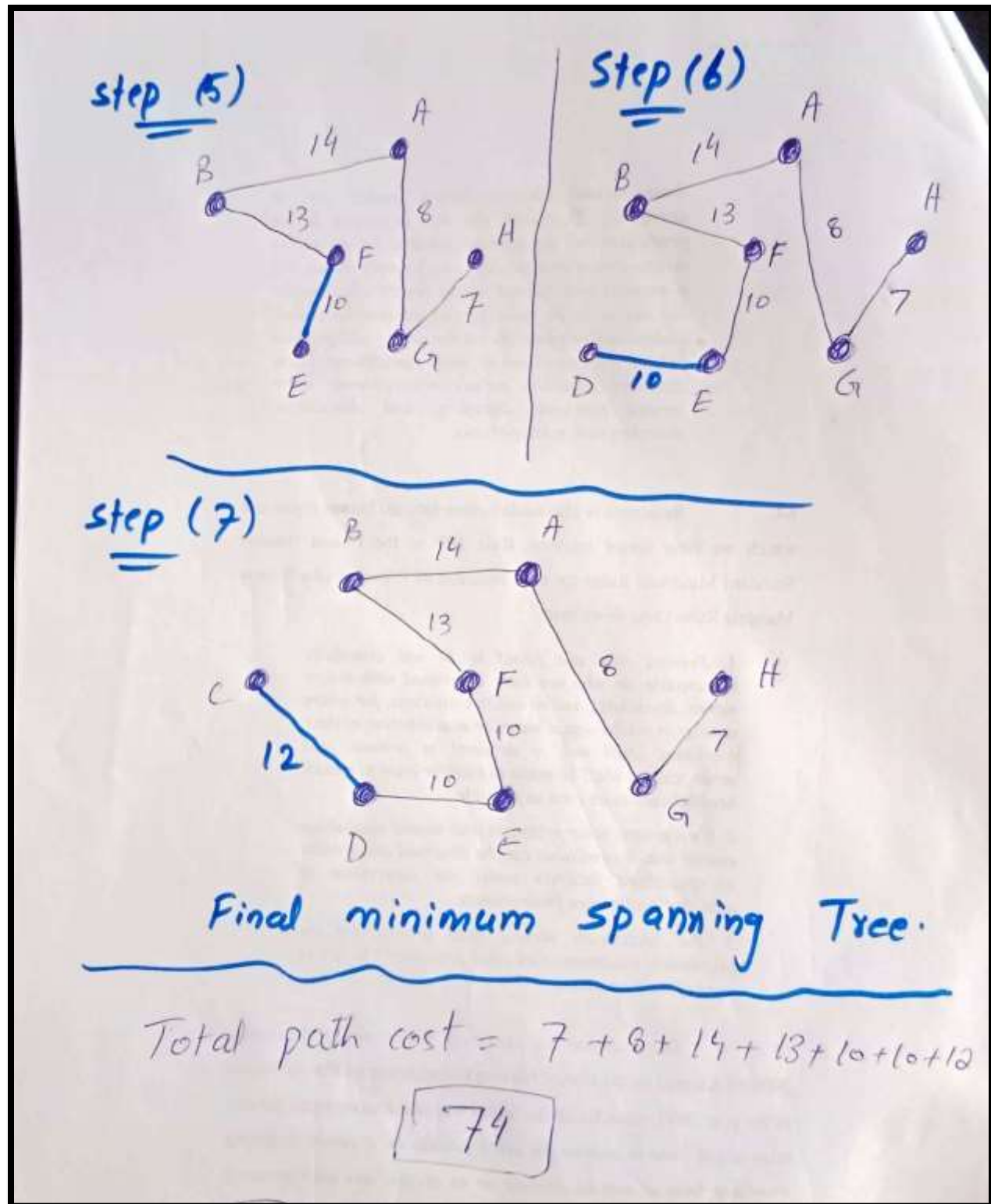
Task 1:

Q1. Apply Prim's algorithm manually on paper add picture and also write its code with output. **5 marks**



DRY RUN:





CODE:

```
def prims_algorithm(graph, start):
    visited = []
    mst_edges = []
    total_weight = 0

    visited.append(start)

    print(f"Starting Prim's Algorithm from node: {start}\n")
    print(f"{'Step':<5} {'Edge':<12} {'Weight':<8} {'Total Weight'}")
    print("-" * 45)

    # We need to visit all nodes (total nodes - 1 edges in MST)
    while len(visited) < len(graph):
        min_weight = None
        min_edge = None

        # Look at all visited nodes
        for node in visited:
            # Check all their neighbors
            for weight, neighbor in graph[node]:
                # Only consider unvisited neighbors
                if neighbor not in visited:
                    if min_weight is None or weight < min_weight:
                        min_weight = weight
                        min_edge = (node, neighbor, weight)

        # Add the cheapest edge found
        if min_edge:
            u, v, w = min_edge
            visited.append(v)
            mst_edges.append(min_edge)
            total_weight += w
            print(f"{'len(mst_edges)':<5} {u}-{v:<10} {w:<8} {'total_weight'}")

    print("-" * 45)
    print(f"\nMST Edges:")
    for u, v, w in mst_edges:
        print(f"  {u} -- {v} (weight: {w})")
    print(f"\nTotal MST Weight: {total_weight}")

    return mst_edges, total_weight
```

```
# — dictionary of the graph
graph = {
    'A': [(14, 'B'), (8, 'G'), (21, 'F')],
    'B': [(14, 'A'), (26, 'H'), (13, 'F'), (14, 'D'), (15, 'C')],
    'C': [(15, 'B'), (12, 'D')],
    'D': [(12, 'C'), (14, 'B'), (10, 'E'), (12, 'F')],
    'E': [(10, 'D'), (10, 'F'), (14, 'G')],
    'F': [(21, 'A'), (13, 'B'), (12, 'D'), (10, 'E')],
    'G': [(7, 'H'), (8, 'A'), (14, 'E'), (33, 'C')],
    'H': [(26, 'B'), (7, 'G')],
}

prims_algorithm(graph, start='A')
```

OUTPUT:

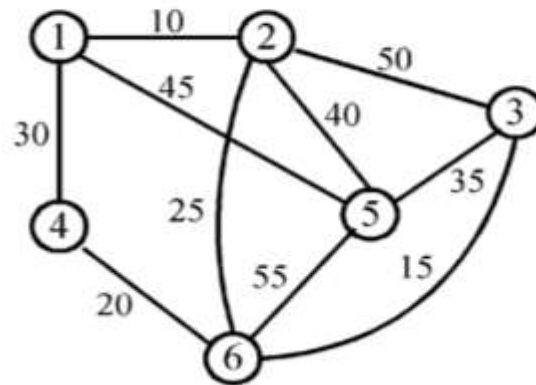
```
/AI/Assignment 3 prims,kruskal and genetic algorithms/primsAlgorithm.py"
Starting Prim's Algorithm from node: A

Step  Edge      Weight  Total Weight
-----
1     A-G        8        8
2     G-H        7       15
3     A-B       14       29
4     B-F       13       42
5     F-E       10       52
6     E-D       10       62
7     D-C       12       74
-----

MST Edges:
A -- G (weight: 8)
G -- H (weight: 7)
A -- B (weight: 14)
B -- F (weight: 13)
F -- E (weight: 10)
E -- D (weight: 10)
D -- C (weight: 12)
Total MST Weight: 74
```

Task :2

Q2. Apply Kruskal's algorithm manually on paper add picture and also write its code with output. 5 marks



DRY RUN:

Kruskal Algorithm:

Step ① writing All nodes in Ascending order Edges.

1:2 → 10

3:6 → 15

4:6 → 20

2:6 → 25

~~1:4 → 30~~

3:5 → 35

~~2:5 → 40~~

~~2:3 → 50~~

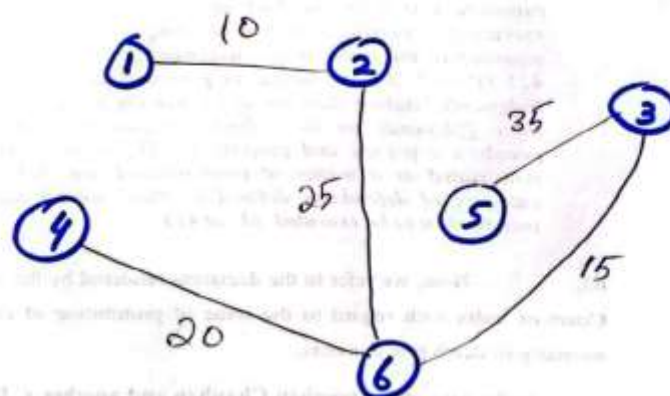
~~1:5 → 45~~

~~2:3 → 50~~

5:6 → 55

total cost = 105

Step ② starting from shortest Edge drawing the graph.



CODE:

```
def kruskals_algorithm(graph):

    # Step 1: Collect all unique edges
    edges = []
    seen = set()

    for node in graph:
        for weight, neighbor in graph[node]:
            edge = tuple(sorted((node, neighbor)))
            if edge not in seen:
                seen.add(edge)
                edges.append((weight, edge[0], edge[1]))

    # Step 2: Sort edges by weight
    edges.sort()

    # Step 3: Union-Find (no library)
    parent = {node: node for node in graph}
    rank = {node: 0 for node in graph}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def union(x, y):
        rx, ry = find(x), find(y)
        if rx == ry:
            return False # same group → cycle
        if rank[rx] < rank[ry]:
            rx, ry = ry, rx
        parent[ry] = rx
        if rank[rx] == rank[ry]:
            rank[rx] += 1
        return True

    # Step 4: Pick edges greedily
    mst_edges = []
    total_weight = 0

    print("All edges sorted by weight:")
    for w, u, v in edges:
```

```

        print(f" {u}-{v} : {w}")

    print(f"\n{'Step':<5} {'Edge':<10} {'Weight':<10} {'Cycle?':<10} {'Total
Weight'}")
    print("-" * 50)

    step = 0
    for weight, u, v in edges:
        step += 1
        if union(u, v):
            mst_edges.append((u, v, weight))
            total_weight += weight
            print(f"{step:<5} {u}-{v:<8} {weight:<10} {'No':<10} {total_weight}")
        else:
            print(f"{step:<5} {u}-{v:<8} {weight:<10} {'Yes - Skip':<10} -")

    if len(mst_edges) == len(graph) - 1:
        break

    print("-" * 50)
    print(f"\nMST Edges:")
    for u, v, w in mst_edges:
        print(f" {u} -- {v} (weight: {w})")
    print(f"\nTotal MST Weight: {total_weight}")

    return mst_edges, total_weight

graph = {
    1: [(10, 2), (30, 4), (45, 5)],
    2: [(10, 1), (50, 3), (40, 5), (25, 6)],
    3: [(50, 2), (35, 5), (15, 6)],
    4: [(30, 1), (20, 6)],
    5: [(45, 1), (40, 2), (35, 3), (55, 6)],
    6: [(25, 2), (15, 3), (20, 4), (55, 5)],
}

kruskals_algorithm(graph)

```

OUTPUT:


```
PS D:\5th Semester data\AI\Assignment 3 prims,kruskal and genetic algorithms>
c algorithms/kruskalAlgorithm.py"
```

```
All edges sorted by weight:
```

```
1-2 : 10
3-6 : 15
4-6 : 20
2-6 : 25
1-4 : 30
3-5 : 35
2-5 : 40
1-5 : 45
2-3 : 50
5-6 : 55
```

Step	Edge	Weight	Cycle?	Total Weight
1	1-2	10	No	10
2	3-6	15	No	25
3	4-6	20	No	45
4	2-6	25	No	70
5	1-4	30	Yes - Skip	-
6	3-5	35	No	105

```
MST Edges:
```

```
1 -- 2 (weight: 10)
3 -- 6 (weight: 15)
4 -- 6 (weight: 20)
2 -- 6 (weight: 25)
3 -- 5 (weight: 35)
```

```
Total MST Weight: 105
```

Task :3

Solving the 8-Queens Puzzle via Evolutionary Computing.

CODE:

```
import copy
import random
from math import exp
import numpy as np

def make_board():
    rows, cols = (board_size, board_size)
    board=[]
    for i in range(rows):
        col=[]
        for j in range(cols):
            col.append("*")
        board.append(col)
    return board

def placeQueens(board):
    i=0
    while(i<board_size):
        row = random.randint(0, board_size-1)
        if board[row][i]!="Q":
            board[row][i]="Q"
            i+=1
    return board

def print_board(board):
    for i in range(board_size):
        for j in range(board_size):
            print(board[i][j], end=' ')
        print()

def getQueens(board, board_size):
    queen_positions=[]
    for i in range(board_size):
        for j in range(board_size):
            if board[i][j]=="Q":
                temp=i,j
                queen_positions.append(temp)
    return queen_positions

def make_chromosome(board):
    temp=[]
    for i in range(board_size):
        for j in range(board_size):
```

```

        if board[j][i]=="Q":
            temp.append(j)
        return temp

def generate_population(size):
    populat =[]
    for i in range(size):
        b = make_board()
        temp =placeQueens(b)
        temp2 = make_chromosome(temp)
        if temp2 not in populat:
            populat.append(temp2)
    return populat

import numpy.random as npr
def selectOne(fit):
    max=sum([c for c in fit])
    selection_probs=[c/max for c in fit]
    return fit.index(fit[npr.choice(len(fit), p=selection_probs)])

def selection(population, fit):
    new_p1 = copy.deepcopy(population)
    new_f1 = copy.deepcopy(fit)
    index1 = selectOne(new_f1)
    c1, f1=new_p1[index1], new_f1[index1]
    index2 = selectOne(new_f1)
    c2, f2=new_p1[index2], new_f1[index2]
    return c1, f1, c2, f2

def crossover(c1, c2):
    ra=np.random.randint(0,4)
    c3=c1[:ra]+c2[ra:]
    c4=c2[:ra]+c1[ra:]
    return c3, c4

def mutation(of_Spring, mutRate):
    for i in range(len(of_Spring)):
        if random.random() < mutRate:
            of_Spring[i] = random.randint(0, board_size - 1)
    return of_Spring

def Evaluate(of_Sp, Of_Fit, parent, p_fit):
    if Of_Fit >= p_fit:
        return of_Sp, Of_Fit
    else:

```

```
    return parent, p_fit

def objective_function(board):
    queen_pos=getQueens(board, board_size)
    h_hit=0
    d_hit=0
    for i in range(0, len(queen_pos)):
        for j in range(i, len(queen_pos)):
            if i != j:
                if queen_pos[i][0]==queen_pos[j][0]:
                    h_hit+=1
                if abs(queen_pos[i][0] - queen_pos[j][0]) == abs(queen_pos[i][1] -
queen_pos[j][1]):
                    d_hit+=1
    return h_hit+d_hit

def fitness_from_chrom(chrom):
    attacks = 0
    n = len(chrom)
    for i in range(n):
        for j in range(i+1, n):
            if chrom[i] == chrom[j]:
                attacks += 1
            if abs(chrom[i]-chrom[j]) == abs(i-j):
                attacks += 1
    max_pairs = n*(n-1)//2
    return max_pairs - attacks

def chrom_to_board(chrom):
    board = make_board()
    for col, row in enumerate(chrom):
        board[row][col] = "Q"
    return board

def run_ga():
    global board_size
    board_size = 8
    POP_SIZE      = 100
    MAX_GEN       = 500
    MUT_RATE      = 0.1
    perfect_score = board_size*(board_size-1)//2
    print("="*55)
    print(f"  N-Queen GA Example    (board size = {board_size})")
    print("="*55)
    population = generate_population(POP_SIZE)
```

```
print(f"\nStep 1 – Initial Population ({len(population)} chromosomes)")
print("  First 4 chromosomes:")
for ch in population[:4]:
    print("    ", ch)

print("\nOne random board from the population:")
print_board(chrom_to_board(population[0]))
best_chrom = None
best_fit = -1

for gen in range(MAX_GEN):
    fit_scores = [fitness_from_chrom(c) for c in population]
    gen_best_fit = max(fit_scores)
    gen_best_chrom = population[fit_scores.index(gen_best_fit)]

    if gen_best_fit > best_fit:
        best_fit = gen_best_fit
        best_chrom = gen_best_chrom[:]
    if best_fit == perfect_score:
        print(f"\n  ✓ Perfect solution found at generation {gen}!")
        break

    new_population = []
    while len(new_population) < POP_SIZE:
        # Selection
        c1, f1, c2, f2 = selection(population, fit_scores)
        # Crossover
        c3, c4 = crossover(c1, c2)
        # Mutation
        c3 = mutation(c3, MUT_RATE)
        c4 = mutation(c4, MUT_RATE)
        # Evaluate
        c3, f3 = Evaluate(c3, fitness_from_chrom(c3), c1, f1)
        c4, f4 = Evaluate(c4, fitness_from_chrom(c4), c2, f2)
        new_population.extend([c3, c4])

    population = new_population[:POP_SIZE]

    if gen % 50 == 0:
        attacks = perfect_score - best_fit
        print(f"  Gen {gen:4d} | Best fitness:
{best_fit}/{perfect_score} | Attacks: {attacks}")

    return gen
```

```
def run_10_times():
    results = []
    for i in range(10):
        gen_count = run_ga()
        results.append(gen_count)
        print(f"Run {i+1:2d}: solved in {gen_count} generations")

    print(f"\nAverage generations: {sum(results)/len(results):.1f}")
    print(f"Min: {min(results)} | Max: {max(results)}")

if __name__ == "__main__":
    run_10_times()
```

OUTPUT: running 10 times using a loop

```
PS D:\5th Semester data\AI\Assignment 3 prims,kruskal and genetic algorithms>
/AI/Assignment 3 prims,kruskal and genetic algorithms/geneticAlgorithm.py"
=====
N-Queen GA Example (board size = 8)
=====

Step 1 - Initial Population (100 chromosomes)
First 4 chromosomes:
[5, 5, 1, 3, 2, 1, 7, 7]
[6, 6, 6, 0, 1, 2, 0, 7]
[0, 6, 6, 1, 7, 3, 1, 0]
[5, 0, 3, 2, 6, 4, 6, 6]

One random board from the population:
* * * * *
* * Q * * Q *
* * * * Q *
* * * Q *
* * * * *
Q Q * * *
* * * * *
* * * * * Q Q

Gen 0 | Best fitness: 25/28 | Attacks: 3
Gen 50 | Best fitness: 27/28 | Attacks: 1
Gen 100 | Best fitness: 27/28 | Attacks: 1
Gen 150 | Best fitness: 27/28 | Attacks: 1
Gen 200 | Best fitness: 27/28 | Attacks: 1
Gen 250 | Best fitness: 27/28 | Attacks: 1
Gen 300 | Best fitness: 27/28 | Attacks: 1
Gen 350 | Best fitness: 27/28 | Attacks: 1
Gen 400 | Best fitness: 27/28 | Attacks: 1
Gen 450 | Best fitness: 27/28 | Attacks: 1
Run 1: solved in 499 generations
```

Step 1 – Initial Population (100 chromosomes)

First 4 chromosomes:

[7, 7, 0, 1, 7, 1, 2, 6]

[6, 7, 1, 7, 4, 5, 7, 3]

[7, 3, 7, 2, 5, 0, 7, 5]

[7, 7, 5, 1, 2, 2, 0, 4]

One random board from the population:

* * Q * * * *

* * * Q * Q * *

* * * * * Q *

* * * * * *

* * * * * *

* * * * * *

* * * * * Q

Q Q * * Q * *

Gen 0 | Best fitness: 25/28 | Attacks: 3

✓ Perfect solution found at generation 48!

Run 2: solved in 48 generations

Step 1 – Initial Population (100 chromosomes)

First 4 chromosomes:

[1, 5, 6, 4, 0, 0, 0, 0]

[3, 3, 7, 4, 0, 7, 5, 5]

[5, 0, 6, 2, 2, 5, 0, 2]

[2, 4, 6, 7, 5, 3, 5, 0]

One random board from the population:

* * * * Q Q Q Q

Q * * * * * *

* * * * * *

* * * * * *

* * * Q * * * *

* Q * * * * *

* * Q * * * *

* * * * * *

Gen 0 | Best fitness: 25/28 | Attacks: 3

Gen 50 | Best fitness: 27/28 | Attacks: 1

✓ Perfect solution found at generation 61!

Run 3: solved in 61 generations

```

Step 1 – Initial Population (100 chromosomes)
First 4 chromosomes:
[1, 2, 7, 7, 5, 7, 7, 7]
[0, 3, 6, 2, 4, 6, 1, 4]
[0, 5, 5, 6, 0, 0, 0, 1]
[1, 5, 3, 2, 0, 6, 2, 1]

One random board from the population:
* * * * *
Q * * * *
* Q * * *
* * * * *
* * * * *
* * * Q *
* * * * *
* * Q Q * Q Q
Gen    0 | Best fitness: 24/28 | Attacks: 4

✓ Perfect solution found at generation 27!
Run 4: solved in 27 generations

```

```

Step 1 – Initial Population (100 chromosomes)
First 4 chromosomes:
[0, 5, 0, 6, 7, 4, 4, 0]
[4, 0, 5, 4, 4, 3, 1, 0]
[2, 0, 2, 4, 2, 5, 4, 2]
[4, 1, 7, 2, 2, 3, 2, 1]

One random board from the population:
Q * Q * * * Q
* * * * *
* * * * *
* * * * *
* * * * Q Q *
* Q * * * *
* * * Q * *
* * * * Q *
Gen    0 | Best fitness: 24/28 | Attacks: 4

✓ Perfect solution found at generation 35!
Run 5: solved in 35 generations

```


Step 1 – Initial Population (100 chromosomes)

First 4 chromosomes:

[4, 7, 3, 6, 2, 6, 1, 5]

[5, 3, 4, 7, 0, 3, 5, 5]

[1, 1, 0, 5, 3, 4, 4, 3]

[6, 6, 0, 5, 2, 0, 6, 2]

One random board from the population:

```

* * * * *
* * * * * Q
* * * * Q *
* * Q * *
* Q * *
Q * * *
* * * * Q
* * Q * Q
* Q * *

```

Gen	0	Best fitness: 25/28	Attacks: 3
Gen	50	Best fitness: 27/28	Attacks: 1
Gen	100	Best fitness: 27/28	Attacks: 1
Gen	150	Best fitness: 27/28	Attacks: 1
Gen	200	Best fitness: 27/28	Attacks: 1
Gen	250	Best fitness: 27/28	Attacks: 1
Gen	300	Best fitness: 27/28	Attacks: 1
Gen	350	Best fitness: 27/28	Attacks: 1
Gen	400	Best fitness: 27/28	Attacks: 1
Gen	450	Best fitness: 27/28	Attacks: 1

Run 6: solved in 499 generations

Step 1 – Initial Population (100 chromosomes)

First 4 chromosomes:

[3, 3, 0, 3, 7, 5, 6, 1]

[3, 5, 0, 6, 4, 6, 5, 3]

[1, 4, 4, 4, 6, 2, 0, 6]

[3, 0, 4, 7, 1, 1, 5, 6]

One random board from the population:

```

* * Q * *
* * * * * Q
* * * * *
Q Q * Q *
* * * *
* * * * Q
* * * * Q
* * * Q

```

Gen	0	Best fitness: 26/28	Attacks: 2
Gen	50	Best fitness: 27/28	Attacks: 1
Gen	100	Best fitness: 27/28	Attacks: 1
Gen	150	Best fitness: 27/28	Attacks: 1
Gen	200	Best fitness: 27/28	Attacks: 1
Gen	250	Best fitness: 27/28	Attacks: 1
Gen	300	Best fitness: 27/28	Attacks: 1
Gen	350	Best fitness: 27/28	Attacks: 1
Gen	400	Best fitness: 27/28	Attacks: 1

✓ Perfect solution found at generation 401!

Run 7: solved in 401 generations

```

Step 1 - Initial Population (100 chromosomes)
First 4 chromosomes:
[7, 7, 0, 7, 5, 5, 7, 3]
[0, 3, 1, 0, 4, 2, 0, 3]
[2, 0, 5, 6, 0, 7, 6, 5]
[5, 2, 1, 6, 4, 3, 7, 3]

One random board from the population:
* * Q * * * * *
* * * * *
* * * * *
* * * * * Q
* * * * *
* * * Q Q *
* * * * *
Q Q * Q * * Q *
Gen    0 | Best fitness: 25/28 | Attacks: 3
Gen   50 | Best fitness: 27/28 | Attacks: 1
Gen  100 | Best fitness: 27/28 | Attacks: 1

✓ Perfect solution found at generation 149!
Run  8: solved in 149 generations

```

```

Step 1 - Initial Population (100 chromosomes)
First 4 chromosomes:
[7, 0, 0, 3, 6, 3, 6, 0]
[2, 7, 2, 6, 1, 6, 7, 3]
[7, 4, 3, 6, 1, 4, 2, 1]
[0, 4, 0, 1, 0, 3, 0, 3]

One random board from the population:
* Q Q * * * * Q
* * * * *
* * * * *
* * * Q * Q *
* * * * *
* * * * *
* * * Q * Q *
Q * * * * *
Gen    0 | Best fitness: 25/28 | Attacks: 3
Gen   50 | Best fitness: 26/28 | Attacks: 2
Gen  100 | Best fitness: 26/28 | Attacks: 2
Gen  150 | Best fitness: 26/28 | Attacks: 2
Gen  200 | Best fitness: 26/28 | Attacks: 2
Gen  250 | Best fitness: 26/28 | Attacks: 2
Gen  300 | Best fitness: 26/28 | Attacks: 2
Gen  350 | Best fitness: 27/28 | Attacks: 1
Gen  400 | Best fitness: 27/28 | Attacks: 1
Gen  450 | Best fitness: 27/28 | Attacks: 1
Run  9: solved in 499 generations

```

Step 1 – Initial Population (100 chromosomes)

First 4 chromosomes:

[0, 6, 5, 6, 2, 0, 5, 4]

[4, 5, 0, 2, 5, 2, 4, 6]

[6, 6, 3, 2, 3, 4, 0, 5]

[1, 6, 3, 0, 2, 7, 1, 6]

One random board from the population:

```
Q * * * * Q * *
* * * * *
* * * * Q * *
* * * * *
* * * * * * Q
* * Q * * * Q *
* Q * Q * * *
```

Gen	0	Best fitness: 26/28	Attacks: 2
Gen	50	Best fitness: 27/28	Attacks: 1
Gen	100	Best fitness: 27/28	Attacks: 1
Gen	150	Best fitness: 27/28	Attacks: 1
Gen	200	Best fitness: 27/28	Attacks: 1
Gen	250	Best fitness: 27/28	Attacks: 1
Gen	300	Best fitness: 27/28	Attacks: 1
Gen	350	Best fitness: 27/28	Attacks: 1
Gen	400	Best fitness: 27/28	Attacks: 1
Gen	450	Best fitness: 27/28	Attacks: 1

Run 10: solved in 499 generations

FINAL RESULT:

```
Average generations: 271.7
Min: 27 | Max: 499
PS D:\5th Semester data\AI\Assignment 3 prims,kruskal and genetic algorithms> |
```

END,,,